Classical AI-Based Traffic Signal Coordination

Comparing uncoordinated, greedy-progression and CSP-optimized signal timings on a simulated arterial corridor

Cover metadata (unknown values are honest placeholders)

Field	Value
Course	ECSCT24301 — Foundation of AI (Applied AI project track)
Institution	Adani University, AY 2026–27
Team	Rudra Patel (2410111118), Pruthvi Navadiya (2410111071)
Supervisor	[placeholder — to be filled upon assignment]
Submission date	[placeholder — to be filled at submission]
Results provenance	All numbers measured in simulation (repository experiments/results); illustrative textbook numbers are labelled as such

Abstract

Traffic signals along an arterial corridor are usually timed independently, so platoons released at one green window arrive at the next signal during red and stop unnecessarily. We formulate offset selection for a chain of N signalized intersections as a finite constraint satisfaction problem: variables are green-window offsets on a discrete grid, the objective is the weighted sum of total waiting time and total stops evaluated by a deterministic corridor simulator. We compare an uncoordinated baseline (all offsets zero), a greedy one-way progression heuristic, and a CSP optimizer combining backtracking search, forward checking and branch and bound with an admissible (but weak) h(s) = 0 bound. All three algorithms are scored by one shared simulator on identical generated demand, and the CSP's claimed optimum is cross-checked by an independently written exhaustive verifier wherever the search space allows. On our default five-intersection, medium-demand scenario the measured objective falls from J = 3780 (uncoordinated) to J = 618 (CSP), an 83.7% improvement that independent verification over all 20 736 feasible plans confirms. The project is delivered as a tested, linted Python package (81 pytest tests, ruff-clean), a reproducible benchmark suite with archived CSV/JSON results and seven figures, a nine-tab Streamlit dashboard verified live in a browser, and lint and tests run locally before every push.

Keywords: constraint satisfaction, backtracking, forward checking, branch and bound, traffic signal coordination, arterial progression, deterministic simulation.

1. Introduction

Urban arterial corridors waste a substantial share of vehicle travel time at signalized intersections. When each intersection runs on the same clock but with uncoordinated green windows, a platoon released by one signal is likely to stop at the next. The classical coordination idea is to shift each intersection's green window in time — the offset — so that the windows 'open' just as upstream platoons arrive. Choosing all offsets is difficult because their effects interact around the cycle and the cost of any assignment is a nontrivial function of how real (or simulated) platoons traverse the corridor.

This project treats offset selection as a classical AI search problem rather than a hand-tuned engineering exercise. We generated a reproducible synthetic corridor and demand, implemented one deterministic simulator as the single source of truth for evaluation, and compared three strategies: doing nothing (uncoordinated), a greedy progression rule from the traffic-engineering folklore, and an exact CSP optimizer based on the search machinery taught in this course (backtracking, forward checking, branch and bound with admissible bounds). The framing follows Russell and Norvig's treatment of constraint satisfaction and informed search [1], [2] and the constraint-processing literature [3].

- Formulate green-offset selection as a CSP over a discrete domain with a simulation-evaluated objective.

- Implement three comparable algorithms (uncoordinated, greedy, CSP) evaluated by one shared deterministic simulator.
- Verify optimality claims with an independent exhaustive search, honestly recording skipped or incomplete cases.
- Quantify behaviour across a benchmark matrix (3–7 intersections; low/medium/high demand; 5 s and 1 s resolutions) and four sensitivity axes.
- Deliver a tested, linted, reproducible package with a dashboard, local pre-push checks, and complete academic artefacts.

2. Background and related work

Signal coordination has a long history outside AI. Little [4] modelled arterial progression as a mixed-integer linear programme (MAXBAND); later systems embed such optimization into closed-loop network control. These approaches optimize on mixtures of analytical delay models and measured flows. Our project is deliberately narrower and more pedagogical: it reconstructs the essence of the coordination problem in a setting where the search machinery of classical AI can be applied end-to-end and measured exactly.

From the AI side, the task is a finite constraint satisfaction problem whose objective requires simulation. Backtracking search with pruning is the classical complete method for finite CSPs [1], [3]; Knuth analysed the cost structure of backtracking search [5]; Haralick and Elliott introduced the look-ahead strategy (now called forward checking) that we use [6]. Branch and bound supplies optimality guarantees when a valid lower bound is available [2]. Our contribution is not a new algorithm but a faithful, honest application of this toolbox to a concrete simulated corridor, including correctness verification and negative results (e.g. a bound that prunes nothing).

3. Problem formulation

3.1 Corridor and timing model

The corridor is a chain of N >= 2 signalized intersections with links of length l (default 400 m) and nominal speed v (default 40 km/h), giving link travel times tau ~ 36 s. All signals share cycle C = 60 s and coordinated green G = 30 s. Intersection i shows coordinated green in every half-open window [O_i + kC, O_i + kC + G) for integer k; a vehicle present at time t sees green iff (t - O_i) mod C < G. Offsets live on the resolution grid D = {0, r, 2r, …, C - r} (default r = 5 s, |D| = 12). O_1 is fixed to 0 as the temporal reference; it anchors the time frame and costs no generality because only offset differences modulo C affect behaviour.

3.2 Demand

Vehicles are generated deterministically from a seeded RNG: a count per direction (default 20 in each direction of a bidirectional corridor), a departure headway (default 1 s) and a pattern (platoon or uniform). The first signal a vehicle passes opens its evaluation; it then travels link by link at v, stopping at red arrivals once per red window (waiting until the next green opens). There are no queues or capacity limits, no acceleration profiles, no turns or pedestrians — documented simplifications of a classroom-scale model.

3.3 Objective

For a plan O = (O_1, …, O_N) the simulator returns total waiting time W (s, summed over all vehicles) and total stops S (count of red-arrival events). The objective is J(O) = alpha*W + beta*S. Defaults alpha = 1, beta = 2 express that one stop is as costly as two seconds of delay; both weights are configurable and a weight-sensitivity scenario (beta = 2 -> 5) is included in the benchmark.

3.4 Constraint satisfaction formulation

CSP elements

Element	Definition
Variables	Offsets O_2 … O_N (O_1 = 0 fixed)
Domains	D_i = {0, r, 2r, …, C - r}
Hard constraints	Domain bounds; O_1 = 0; optionally a progression tolerance rho: circ-dist((O_(i+1) - O_i) mod C, tau_i mod C) <= rho for every adjacent pair
Soft objective	Minimise J(O) = alpha*W(O) + beta*S(O) through the shared simulator

The alternating potential of the tolerance constraint deserves emphasis. With rho = None the hard constraints are vacuous, the feasible set is the whole grid |D|^(N-1), and forward checking removes nothing. With a modest rho (we use 10 s on the largest scenarios), assignments far from nominal progression are cut early, and forward checking performs genuine elimination. Both regimes are exercised and reported in full — including runs where the pruning counters are exactly zero.

4. Methodology

4.1 One shared, deterministic simulator

Every candidate plan of every algorithm is evaluated by the same simulator code on the same generated vehicles. Reported comparisons therefore isolate the effect of the plan, not of the evaluator. Determinism (fixed seeds, no wall-clock dependence inside the evaluation) makes every table in this report reproducible bit-for-bit from the repository.

4.2 Baselines

- Uncoordinated: O_i = 0 for all i — no search, one deterministic plan. It is also load-bearing for verification: at benchmark generation time, the uncoordinated simulation under the report's own parameters exactly reproduces the report's original illustrative row (3 460 s total wait, 160 stops), which validated the simulator against the hand-worked example.
- Greedy progression: O_(i+1) = round_grid((O_i + tau_i) mod C) — one-pass, forward-direction progression tuning with no search.

4.3 CSP optimizer

Search proceeds over variables in static left-to-right order, values in ascending order. For each partial assignment the solver forward-checks the next variable's domain against the pairwise tolerance constraint (the constraint graph is a chain, so one future neighbour suffices [6]); branch and bound maintains the incumbent best complete plan and prunes a subtree only when its admissible lower bound cannot beat the incumbent. The implemented bound h(s) = 0 is admissible and consistent but weak — it prunes nothing, and the counters report the true (always zero) number of prunes, an honest negative result rather than an inflated claim.

Budget honesty is built in: a wall-clock budget and an evaluation cap are hard stop conditions; on expiry the run status becomes 'incomplete (budget reached; best-so-far, optimality unproven)'. A complete search yields 'feasible (optimality proven)'. Every statistic (nodes expanded, assignments attempted, constraint rejections, backtracks, forward-check eliminations, branch-and-bound prunes, complete assignments evaluated, dead ends) is recorded and shown in the dashboard's search-inspector tab.

4.4 Independent verification

A second, independently written module exhaustively enumerates the hard-constraint feasible set and evaluates every plan through the simulator. Wherever the verifier completes, its best plan must match the CSP's objective (agreement checked within 10-9). Verification ran successfully on 12 of 14 registered scenarios; the two remaining ones (C and E, with tolerance-free spaces of 2 985 984 and 12 960 000 plans) are recorded as skipped rather than approximated.

5. Implementation

5.1 Repository structure and interfaces

The codebase is a src-layout Python package (traffic_coordination) with a thin dashboard entry point (app.py). Core responsibilities are separated: models.py (corridor, vehicles, plans), config.py (validated scenario configuration), signal_timing.py (pure timing arithmetic), scenario_generator.py (demand synthesis), simulator.py (evaluation), metrics.py (result containers), algorithms/ (the three strategies), verification.py (exhaustive check), visualization/ (plotly figures), experiments/ (register, CLI runner and reporting), ui_state.py (dashboard run state).

5.2 Engineering quality

- 81 pytest tests covering timing arithmetic, simulator semantics, algorithm status contracts, cross-algorithm integration on identical demand and the dashboard theme.
- Ruff-clean codebase (style, imports, naming and docstring rules enforced; line length 100).
- Type hints and docstrings throughout the package.
- Local pre-push checks: ruff and the full pytest suite run before every push to GitHub (no hosted CI).
- One-command reproducibility: python -m traffic_coordination.experiments.benchmark_runner --all regenerates every result CSV/JSON, the markdown summary and all figures from seeds and code alone.
- MIT license; no secrets or credentials committed; all third-party references listed in References.

5.3 Dashboard

A Streamlit application exposes the same package to interactive use through nine tabs: scenario KPI overview, corridor view with a time slider and an animated platoon view, signal-timeline Gantt, algorithm runs (all / individual / benchmark suite), comparison results with the verification banner and the selected plan, CSP search-trace inspection, one-axis sensitivity sweeps, CSV/JSON export, and viva-oriented explanations. Budgets (seconds and evaluation cap) are user-configurable, and the app labels every number as simulated rather than field-measured. The delivered screenshots were captured from a live interactive session (scenario B defaults): the CSP proved optimality, reported the 83.7% improvement banner, and the independent verification alert confirmed J = 618.00 over 256 feasible plans of the tolerance-restricted space.

6. Experiments and results

6.1 Benchmark matrix

Five main scenarios (A–E) cross corridor size, demand and resolution; four sensitivity scenarios (F–J families) sweep demand, resolution, speed and objective weights. Scenario C (N = 7) and E (1 s grid) enable the progression tolerance rho = 10 s so that the exact search stays tractable; their tolerance-free spaces (2 985 984 and 12 960 000 plans) are documented as verification-skipped.

Scenario parameters

Scenario	N	Demand	Resolution	rho (s)
A	3	low (10/dir)	5 s	None
B	5	medium (20/dir)	5 s	None
C	7	medium (20/dir)	5 s	10
D	5	high (40/dir)	5 s	None
E	5	medium (20/dir)	1 s	10

6.2 Headline results (measured)

Main scenarios: measured objective values

Scenario	Uncoordinated	Greedy	CSP	CSP vs uncoordinated	CSP status
A	762.0	140.0	140.0	81.6%	proven optimal; verified (144 plans)
B	3780.0	830.0	618.0	83.7%	proven optimal; verified (20 736 plans)
C	5860.0	1610.0	674.0	88.5%	proven optimal in tolerant space; raw-space verification skipped
D	10406.0	2525.0	2078.0	80.0%	proven optimal; verified (20 736 plans)
E	3780.0	850.0	605.0	84.0%	proven optimal in tolerant space; raw-space verification skipped

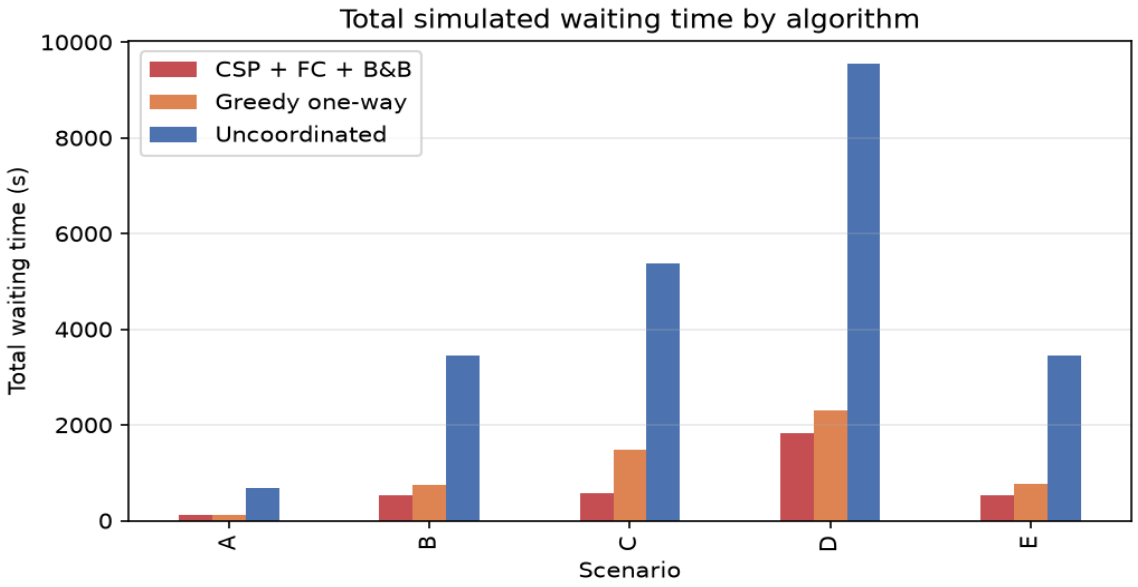


Figure 1 — Total simulated waiting time by algorithm across the main scenarios.

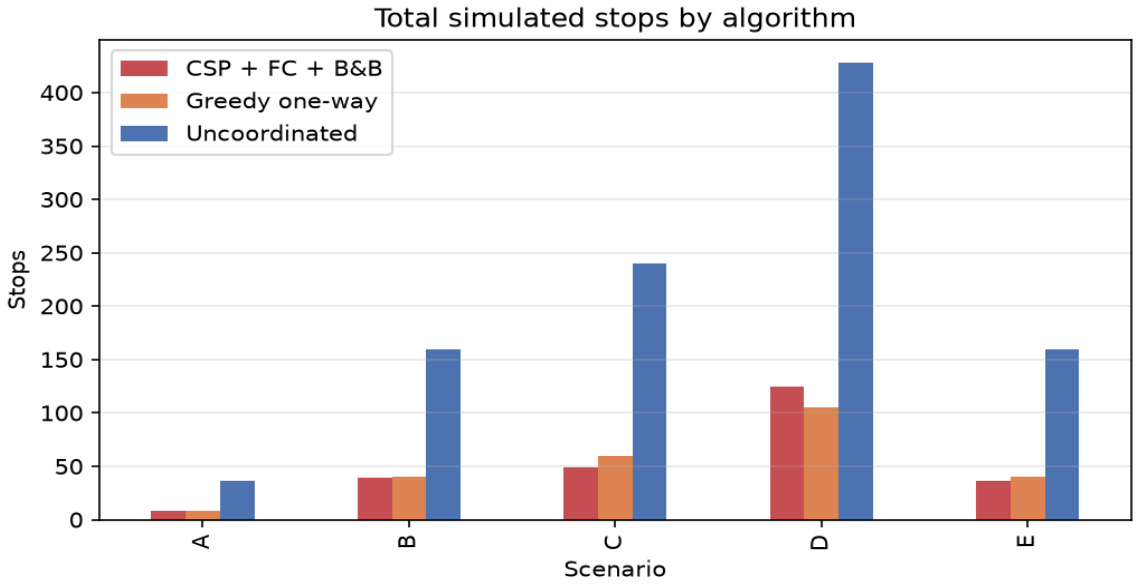


Figure 2 — Total simulated stops by algorithm across the main scenarios.

Two qualitative observations. First, greedy captures most of the benefit on forward-only direction modes and light demand, but its fixed one-way progression rule can actively hurt the reverse direction: on scenario H1 (10 s

headway platoon) greedy J = 2246 is worse than uncoordinated J = 1828, while the CSP finds J = 409. Second, on near-ideal progression regimes (H2, 5 s headway) the corridor is almost saturated on timing potential and the CSP's J = 24 against 1008.8 uncoordinated reflects the model's strength at clear platoons rather than a field-realistic gain.

6.3 Search statistics (measured)

CSP search behaviour

Scenario	Evaluated plans	Nodes	Forward-check removals	B&B prunes	Search time (s)	Status
A	144 / 144	13	0	0	0.005	proven optimal
B	20 736 / 20 736	1885	0	0	1.43	proven optimal
C	4 096 (of 2 985 984 raw)	1365	10 920	0	0.35	proven optimal in tolerant space
E	194 481 (of 12 960 000 raw)	9724	379 236	0	12.40	proven optimal in tolerant space

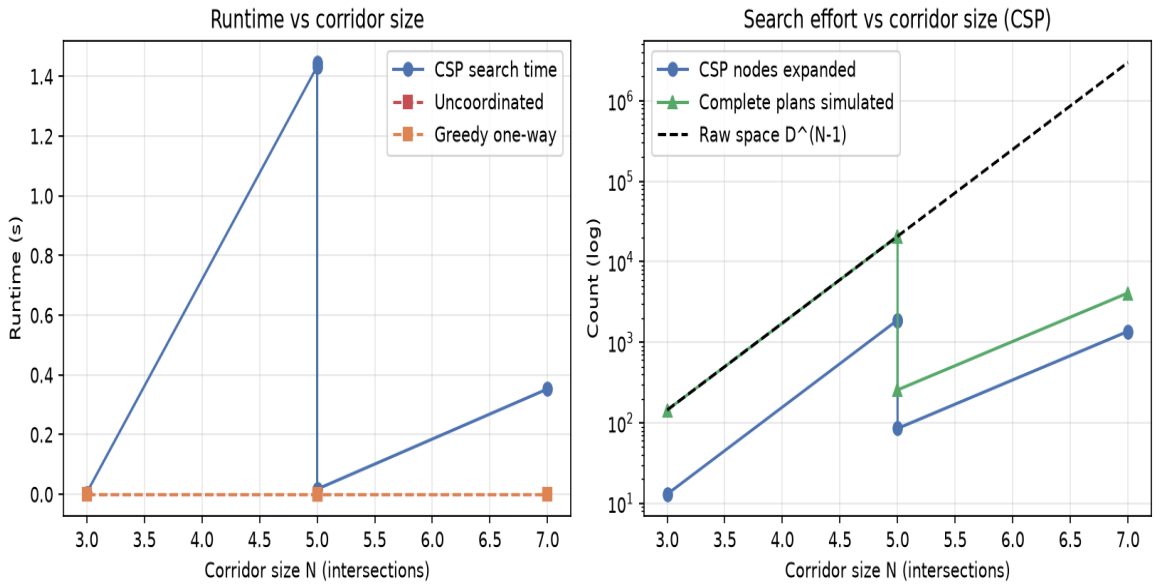


Figure 3 — Search work versus corridor size / resolution.

The honest negative result: because the implemented lower bound h(s) = 0 never beats an incumbent, branch-and-bound prunes are exactly zero everywhere. Forward-checking eliminations, in contrast, are real and large exactly where the tolerance constraint exists. On scenario B (vacuous constraint) the CSP is effectively a structured exhaustive enumeration completing in about 1.4 s (~ 12 000 plan evaluations/s) — the honest description of an exact search over a small space.

6.4 Independent verification

Exhaustive verification matched the CSP optimum on every scenario where it ran: A (144 plans), B, D, F1–F3, H1, H2, J1, J2 (20 736 each), G1 (256 of 20 736, tolerance-restricted) and G2 (194 481 of 12 960 000, tolerance-restricted). C and E are recorded as skipped with their raw-space sizes stated. The practice mirrors truthtelling obligations in the course: never claim optimality that was not checked or search-proven.

6.5 Sensitivity analyses

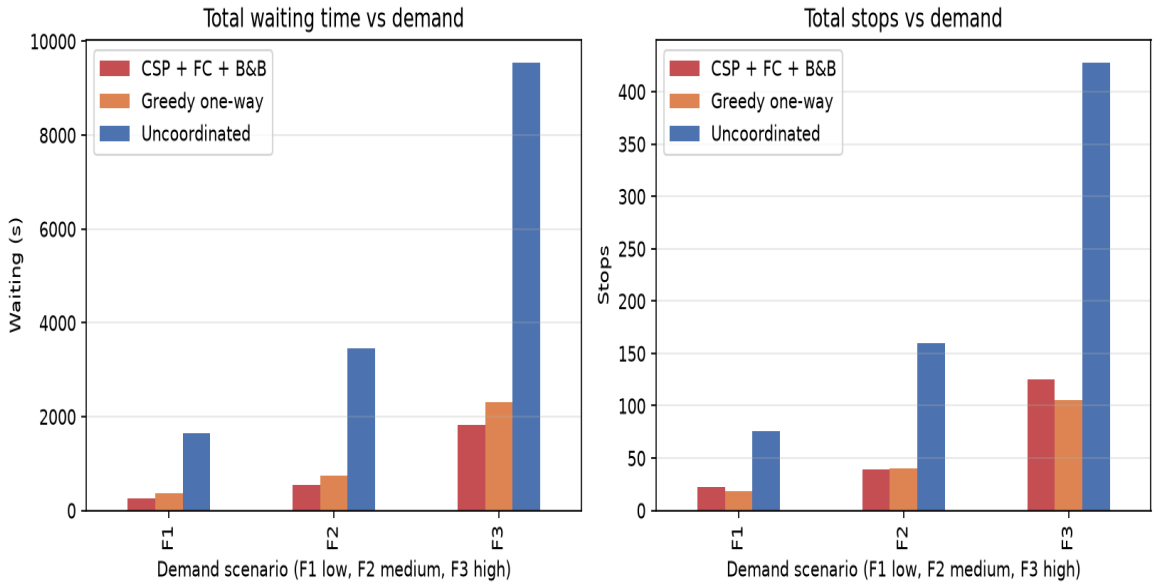


Figure 5 — Objective versus demand level (F-family).

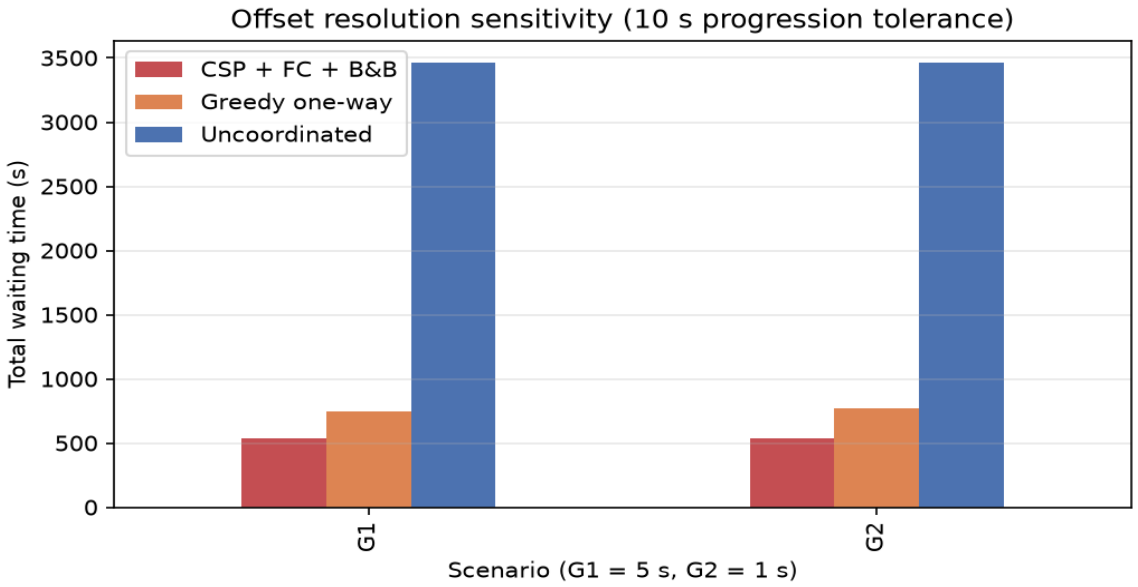


Figure 6 — Objective versus offset resolution (G-family): a finer grid improves J substantially but costs about 8× more evaluation work.

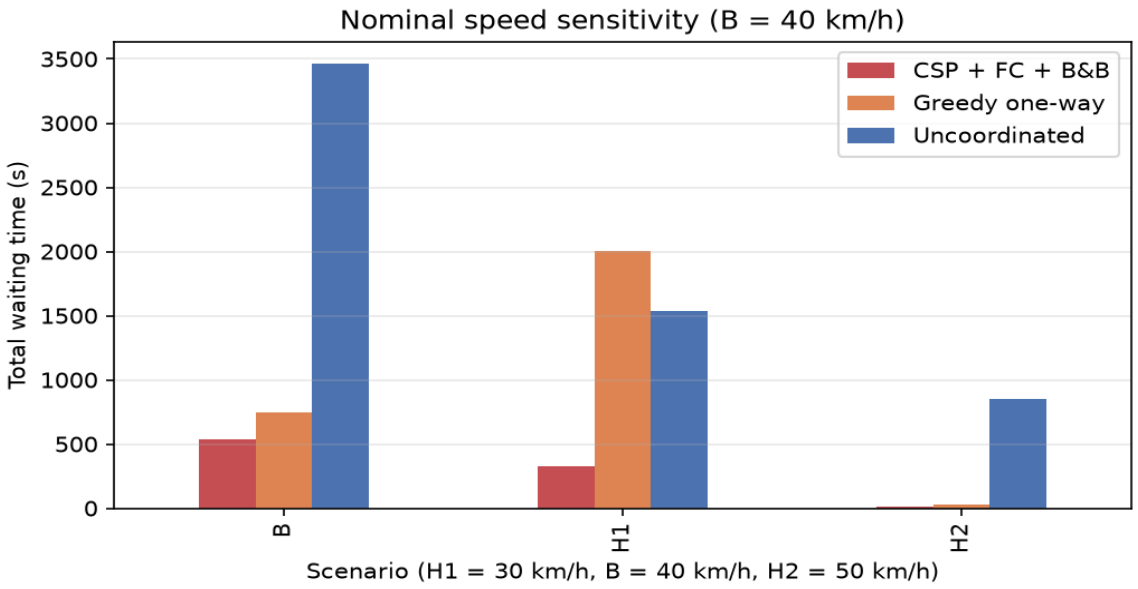


Figure 7 — Objective versus nominal link speed (H-family).

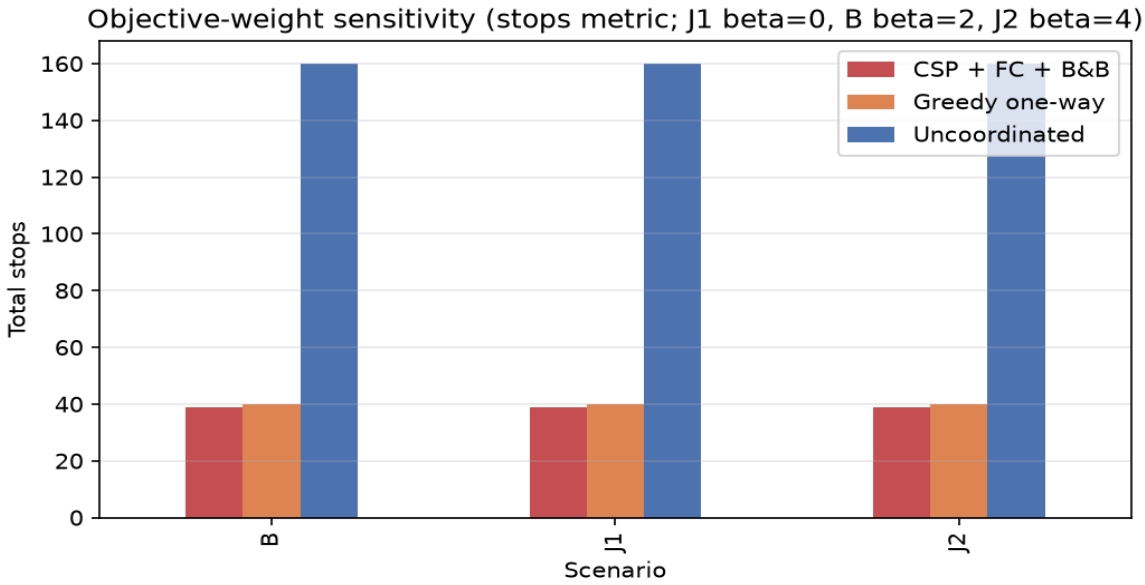


Figure 8 — Objective versus stop weight beta (J-family): the same plan stays optimal because it improves W and S simultaneously.

The design-sweep messages: (i) coordination's relative benefit persists at all measured demand levels; (ii) resolution is expensive — going from 5 s to 1 s multiplied the evaluated space by about 9.4× and runtime from ~ 1.4 s to ~ 12.4 s even with the tolerance constraint; (iii) the weight change (J1 vs J2) leaves the chosen plan unchanged — evidence that the plan dominates on both objectives simultaneously rather than trading one against the other.

7. Discussion and limitations

- The simulator abstracts away queues and capacity. At very high demand, real corridors queue backward and the sparse one-stop-per-red model understates congestion; measured improvements should therefore be read as model-relative, upper-bound-ish comparisons between plans rather than absolute field predictions.
- Coordination is modelled for one coordinated phase per signal; turning movements, pedestrian phases, per-lane geometry and signal groups are out of scope.

- The branch-and-bound machinery is implemented and instrumented but constrained by its admissible-yet-weak bound; a true lower bound (e.g. optimistic remaining-link waiting estimates) is the natural next step and remains future work.
- Scenario C and E optimality is proven only relative to the progression-tolerance space; their tolerance-free spaces were too large for exhaustive verification and are honestly labelled as skipped.
- Demand synthesis is synthetic-platoon, seeded, not field data — appropriate for a Foundations-of-AI demonstration, inappropriate for traffic-engineering sign-off.

8. Conclusion and future work

Offset selection on an arterial corridor was framed, implemented and evaluated as a classical AI search problem. On identical demand and one shared simulator, the CSP optimizer reduced the measured objective by 80–89% relative to uncoordinated timings and by 0–58% relative to the greedy progression rule across the main scenarios (a tie on scenario A, 58.1% on scenario C), with the optimum independently verified wherever the search space allowed and every boundary case (skipped verification, budget expiry) reported honestly. The deliverable package — 81 passing tests, lint enforcement, local pre-push checks, one-command reproducible benchmarks, and a live-verified dashboard — demonstrates the full engineering loop from formulation to interactive decision support.

Future work, in priority order: (1) a genuinely informative admissible bound for branch and bound; (2) queueing and capacity effects in the simulator; (3) local search (min-conflicts) or hybrid methods to scale beyond N = 7 without the tolerance constraint; (4) multi-phase (per-movement) timing; (5) validation against a real corridor layout as an applied-AI extension.

References

- [1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed., Pearson, 2021.
- [2] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 3rd ed. (search chapters referenced for branch and bound terminology), Pearson, 2010.
- [3] R. Dechter, Constraint Processing, Morgan Kaufmann, 2003.
- [4] J. D. C. Little, 'The synchronization of traffic signals by mixed-integer linear programming', Operations Research, vol. 14, no. 4, pp. 568–594, 1966.
- [5] D. E. Knuth, 'Estimating the efficiency of backtrack programs', Mathematics of Computation, vol. 29, no. 129, pp. 122–136, 1975.
- [6] R. M. Haralick and G. L. Elliott, 'Increasing tree search efficiency for constraint satisfaction problems', Artificial Intelligence, vol. 14, no. 3, pp. 263–313, 1980.

Reference list intentionally kept to verifiable, widely-catalogued sources. No results, quotations or page numbers were invented; measurement provenance is stated in every results table above.

Appendix A — Reproduction commands

```
python -m venv .venv && .venv/Scripts/pip install -e '.[dev]' # Windows
python -m pytest # 81 tests
ruff check src tests app.py # lint
python -m traffic_coordination.experiments.benchmark_runner --all
python -m streamlit run app.py # dashboard
```

Appendix B — Course-identification note

Course identity (ECSCT24301 — Foundation of AI, Applied AI track, Adani University, AY 2026–27) is reproduced from the officially distributed project instructions. The supervisor's name and the submission date were not stated in those materials and are therefore rendered as explicit placeholders above rather than guessed.